

END SEM BLUEPRINT

Data Structures & Algorithms – II | BCSE0401 / BCS301

B.Tech 4th Sem · NIET Greater Noida · Units 1–5

What you are actually studying — read this first:

After reading all 5 uploaded files carefully, here is the truth: **All 5 units are DSA-2 (Data Structures and Algorithms-II)** — not a mix with DBMS. Unit 1 = Trees (BST, AVL, Heap). Unit 2 = SQL & Normalization (DBMS — this is the ODD one out, taught as part of a combined course). Unit 3 = Dynamic Programming (Knapsack, LCS, Floyd Warshall, Matrix Chain). Unit 4 = Backtracking & Branch and Bound (N-Queens, Graph Coloring, TSP, Subset Sum). Unit 5 = Advanced Data Structures (Red-Black Trees, B+ Trees, Binomial & Fibonacci Heaps, Tries). This blueprint is built entirely from what is actually in your slides — nothing assumed.

■ 5-Unit Snapshot — Difficulty & Exam Weight

Unit 1	Trees: BST, AVL, Heap Sort, B-Tree	Medium-Hard	■■■■■	Most practised topic — numericals are very doable with practice
Unit 2	DBMS: SQL, Normalization, FDs	Medium	■■■■■	Conceptual but scoring — normalization numericals repeat every year
Unit 3	Dynamic Programming: Knapsack, LCS, Floyd, MCM	Hard	■■■■	Hardest unit — table-filling problems need practice. High reward if mastered.
Unit 4	Backtracking & Branch-Bound: N-Queens, TSP, Graph Coloring	Medium-Hard	■■■■	N-Queens and TSP diagrams appear in almost every paper — very learnable
Unit 5	Advanced DS: Red-Black, B+, Binomial/Fibonacci Heaps, Tries	Hard	■■■	Heaviest theory unit — focus on properties and insertion only. Skip deep proofs.

* Weight estimated from slide assignment frequency, expected-question slides, and MCQ density.

■ UNIT 1 — Trees (BCS301 / BCSE0401)

Unit 1 covers Binary Trees, BST, AVL Trees, B-Trees, Heaps, and Heap Sort. The slides explicitly list "Expected Questions for University Exam" and weekly assignments — these are your clearest signal of what will appear. Traversal construction, AVL rotations, and B-Tree insertion come up in virtually every paper.

■ High Priority — Must do numericals

- Tree traversals: Inorder, Preorder, Postorder — with code
- Construct binary tree from Inorder + Preorder (e.g. Inorder: H K D B I L E A F C M J G)
- Construct binary tree from Inorder + Postorder
- BST: insert 43,10,79,90,12,54,11,9,50 with step-by-step diagram
- BST deletion: all 3 cases (leaf, one child, two children)
- AVL tree insertions with LL / LR / RL / RR rotations
- Create AVL from: a,z,b,y,c,x,d,w,e,v,f (from expected questions)
- B-Tree of order 4 or 5 insertions — from expected questions
- Heap Sort algorithm + trace example A=[7,4,3,1,2]
- Build Max/Min heap + insert + extract-max

■ Quick Wins — Learn definitions only

- Basic terminology: Root, Parent, Child, Sibling, Leaf, Degree, Height, Depth
- Full Binary Tree theorem: $L=I+1$, $N=2I+1$, $N=2L-1$
- Strictly Binary vs Complete Binary vs Extended Binary — 1 line each
- Array vs Linked List representation — just the concept
- BST advantages over arrays ($O(\log n)$ search)
- AVL Balance Factor = $\text{height}(\text{left}) - \text{height}(\text{right})$
- Threaded Binary Tree — definition only (rarely asked deep)
- Priority Queue — just what it is

■ Expected Questions — Unit 1 (directly from slides)

1	Define extended binary tree, full binary tree, strictly binary tree and complete binary tree.	5 M
2	Explain threaded binary tree. Can you find a unique tree when any two traversals are given?	5 M
3	Construct binary tree from: INORDER: H K D B I L E A F C M J G PREORDER: A B D H K E I L C F G J M. Also find Postorder.	10 M
4	What is a B-Tree? Generate B-Tree of order 4 with: a g f b k d h m j e s i r x c l n t u p	10 M
5	Draw binary tree for expression: $A * B - (C + D) * (P / Q)$	7 M
6	What is a height-balanced tree? Why is height balancing required? Create AVL Tree for: a, z, b, y, c, x, d, w, e, v, f	10 M

7	Construct B-tree of order 5 inserting: 3,14,7,1,8,5,11,17,13,6,23,12,20,26,4,16,18,24,25,19. Also delete 6, 23 and 3.	10 M
8	Create BST using 28,56,12,9,87,7,23,19. Delete 56. Show all steps.	10 M
9	What is AVL Tree? Explain LL, LR, RL, RR rotations with diagrams. Create AVL using 39,56,2,9,87,7,23,19.	10 M
10	Explain Heap Sort algorithm. Trace on A=[7,4,3,1,2]. What is its time complexity?	10 M
11	Prove: number of leaves in a non-empty full binary tree is one more than number of internal nodes.	5 M
12	Post Order of BST is 10,9,23,22,27,25,15,95,60,40,29. Find PreOrder traversal.	7 M

UNIT 2 — DBMS: SQL & Normalization (BCSE0402)

Honest note about Unit 2:

Unit 2 is DBMS (a different subject code BCSE0402), not DSA-2. It is taught alongside DSA-2 in 4th Sem. Your slides cover Keys, Constraints, SQL Functions, and Normalization (1NF to BCNF). The expected questions section in the slide explicitly names 16+ university-level questions. **Normalization numericals and candidate key problems are the most frequently asked.**

High Priority — Do all numericals

- All key types with examples: Super, Candidate, Primary, Alternate, Composite, Foreign, Unique
- Normalization: 1NF → 2NF → 3NF → BCNF with step-by-step example
- Functional Dependencies: definition, trivial vs non-trivial, attribute closure (A+)
- Canonical Cover (Minimal Cover) algorithm — very frequently asked
- Finding Candidate Keys from a given set of FDs — numerical
- SQL Aggregate: COUNT, SUM, AVG, MAX, MIN with GROUP BY + HAVING
- WHERE vs HAVING clause — distinction always asked
- SQL Constraints: NULL, NOT NULL, DEFAULT, CHECK with examples

Quick Wins — Definition level only

- Super key vs Candidate key in 2 lines
- String functions: CONCAT, LENGTH, UPPER, LOWER, TRIM
- Math functions: ABS, ROUND, CEIL, FLOOR
- BETWEEN, LIKE, DISTINCT, LIMIT predicates with syntax
- SQL Aliases using AS keyword
- What is a relation / tuple / attribute — basic DBMS terms
- ORDER BY ASC/DESC — just syntax

Expected Questions — Unit 2

1	Explain Super Key, Candidate Key, Primary Key with examples. What is Alternate Key?	7 M
2	Discuss all key types in DBMS with examples (Super, Candidate, Primary, Alternate, Composite, Foreign, Unique).	10 M
3	Given FDs on relation R(A,B,C,D): find all candidate keys using attribute closure. Check normal forms.	10 M
4	Normalize the given relation step by step from 1NF to BCNF. Explain each normal form clearly.	10 M
5	What is Canonical Cover? Construct canonical cover for FD set {A→BC, B→C, A→B, AB→C} and verify.	10 M

6	Explain the difference between WHERE and HAVING clauses with examples. Write SQL for avg salary > 5000.	7 M
7	Demonstrate Aggregate Functions (SUM, AVG, COUNT, MAX, MIN) with GROUP BY. Write SQL queries.	7 M
8	What are constraints in SQL? Explain NULL, NOT NULL, DEFAULT, CHECK with practical examples.	7 M
9	What is Functional Dependency? Explain types. Find attribute closure of A+ given FD set.	7 M
10	Discuss limitations of 3NF and justify the need for BCNF. What is the difference between 3NF and BCNF?	7 M
11	Write SQL queries using LIKE, BETWEEN, DISTINCT, ORDER BY, LIMIT with examples.	5 M

■ UNIT 3 — Dynamic Programming (DSA-2)

Reality check for Unit 3:

Unit 3 is Dynamic Programming — **the hardest unit numerically**. The slides cover: 0/1 Knapsack (table-filling), Floyd-Warshall (all-pairs shortest path), Longest Common Subsequence (LCS), Matrix Chain Multiplication, and Assembly Line Scheduling. Every one of these requires filling a DP table step by step. You CANNOT memorise your way through this — you must practice the table-filling method. But the good news: the algorithm and steps are always the same for each problem.

■ High Priority — Practice table-filling

- 0/1 Knapsack: build $c[i][w]$ table, trace back solution
- Floyd-Warshall: initialize $D[i][j]$ matrix, iterate through each vertex as intermediate
- LCS: build LCS table for two given strings, find length and actual subsequence
- Matrix Chain Multiplication: fill $m[i][j]$ cost table, find optimal parenthesization
- Assembly Line Scheduling: setup, formula $f1(j)$ and $f2(j)$, find minimum time
- DP properties: Optimal substructure + Overlapping subproblems — always asked in theory

■ Quick Wins — Understand the concept

- What is Dynamic Programming? When to use it vs Greedy vs D&C;
- Difference between DP and Greedy method (1 table comparison)
- Memoization vs Tabulation — top-down vs bottom-up
- Time complexity of each algorithm: Knapsack $O(nW)$, LCS $O(mn)$, Floyd $O(V^3)$, MCM $O(n^3)$
- Fractional Knapsack is NOT solved by DP — trick MCQ question
- LCS vs substring distinction

■ Expected Questions — Unit 3

1	Explain 0/1 Knapsack using DP. Given: items with weights [2,3,4,5] and profits [3,4,5,6], max capacity $W=5$. Fill the table and find optimal selection.	10 M
2	Solve Floyd-Warshall All-Pairs Shortest Path on the given graph. Show each step of the D matrix.	10 M
3	Find the LCS of $S1=\{B,C,D,A,A,C,D\}$ and $S2=\{A,C,D,B,A,C\}$ using DP. Show the table and backtrack the solution.	10 M
4	Explain Matrix Chain Multiplication using DP. Find optimal parenthesization for matrices: $A(10 \times 30)$, $B(30 \times 5)$, $C(5 \times 60)$, $D(60 \times 8)$.	10 M
5	What are the properties of a DP problem? Distinguish between Optimal substructure and Overlapping subproblems with examples.	7 M
6	Write the difference between Greedy method and Dynamic Programming. Give one example where Greedy fails but DP works.	7 M

7	Explain Assembly Line Scheduling using DP. What is the recurrence relation? Solve a numerical example.	7 M
8	Write the algorithm for 0/1 Knapsack DP. What is its time and space complexity? Why is Fractional Knapsack NOT a DP problem?	5 M

UNIT 4 — Backtracking & Branch and Bound (DSA-2)

Unit 4 covers Backtracking (N-Queens, Graph Coloring, Sum of Subsets, Hamiltonian Circuit) and Branch and Bound (Travelling Salesman Problem). The slides explicitly list 6 "Expected Questions" — every one of them is listed below. N-Queens and TSP are the most common 10-mark questions in this unit.

■ High Priority — Draw state-space trees

- N-Queens problem: 4-Queen solution with state-space tree (from expected questions)
- 8-Queen problem: at least one solution path
- Travelling Salesman Problem (TSP) using Branch & Bound: cost matrix reduction, node expansion
- Graph Coloring: concept + algorithm + example with m colors
- Sum of Subsets: given $S=\{3,4,5,6\}$ $X=9$ — backtracking tree (in slides)
- Hamiltonian Cycle: definition + when it exists
- Floyd-Warshall vs Backtracking — know when to use which

■ Quick Wins — Theory only

- What is Backtracking? Definition + when to use
- State-space tree — definition and how it is constructed (DFS)
- Three types of backtracking problems: Decision, Optimisation, Enumeration
- What is Branch and Bound? How it differs from backtracking
- Pruning: bounding function + when to prune a node
- Subset Sum problem statement in 3 lines
- Greedy vs DP vs Backtracking — 1-line comparison table

■ Expected Questions — Unit 4 (directly from slides)

1	Solve the Travelling Salesman Problem using Branch and Bound on the given cost matrix. Show expansion for each vertex.	10 M
2	Write short notes on: (i) Graph Coloring (ii) Subset Sum Problem (iii) Hamiltonian Cycle	10 M
3	Find one optimal solution for: (a) 4-Queen problem on 4x4 chessboard (b) 8-Queen problem on 8x8 chessboard using Backtracking.	10 M
4	Write the difference between the Greedy method and Dynamic Programming.	5 M
5	Explain the term State-Space Search Tree with a suitable example.	7 M
6	Explain Floyd-Warshall algorithm and also analyze its time complexity.	7 M
7	What is Backtracking? Explain with example. What is a state-space tree? How is it constructed?	7 M
8	Given $S=\{3,4,5,6\}$ and $X=9$, solve Subset Sum using Backtracking. Draw the implicit binary tree.	10 M
9	Explain Graph Coloring Problem using Backtracking. Write the algorithm and apply for 4-vertex graph with $m=3$ colors.	7 M

■ UNIT 5 — Advanced Data Structures (DSA-2)

Honest assessment of Unit 5:

Unit 5 (Red-Black Trees, B-Trees, B+ Trees, Binomial Heaps, Fibonacci Heaps, Tries) is the most theoretically heavy unit. The slides are dense (176 slides) but have **no explicit "Expected Questions" section** — unlike Units 1 and 4. Based on the slide content and standard exam patterns: Focus on RB Tree properties + insertion, B+ Tree vs B-Tree difference, and Tries (prefix trees). Binomial and Fibonacci Heap deep insertion algorithms are low-priority unless your teacher specifically said so. **Do not spend 3 days on this unit — 1.5 days maximum.**

■ High Priority — Focus here

- Red-Black Tree: 5 properties (every node red/black, root black, leaves black, red parent has black children, equal black-height)
- RB Tree: why it is needed ($O(\log n)$ guaranteed vs BST $O(n)$ worst case)
- RB Tree height theorem: $\text{height} \leq 2\log(n+1)$
- RB Tree rotations: Left-Rotate and Right-Rotate — how they work
- B-Tree vs B+ Tree: key differences (data in leaves only for B+, linked leaf nodes)
- Tries: definition, insert/search operations, use cases (autocomplete, dictionary)
- Binomial Heap: definition + properties of binomial trees B_k

■ Quick Wins — Know enough to write 5 marks

- Black-height: $bh(x)$ = number of black nodes from x to leaf (not counting x)
- RB Tree sentinel $nil[T]$ — what it is and why used
- Fibonacci Heap: better amortized complexity than binary heap (decrease-key $O(1)$)
- B-Tree of order m : each non-root has $\lceil m/2 \rceil - 1$ to $m-1$ keys
- B+ Tree: all data records at leaf level, leaves linked as a list
- Tries: each edge represents a character, prefix tree for string searching

■ Most Likely Questions — Unit 5

1	What is a Red-Black Tree? List all 5 properties. Prove that a Red-Black Tree with n nodes has height at most $2\log(n+1)$.	10 M
2	Explain Left-Rotate and Right-Rotate operations in Red-Black Trees with algorithm and diagram.	7 M
3	What is a B+ Tree? How is it different from a B-Tree? Draw an example and explain advantages of B+ Tree.	7 M
4	What is a Trie? Explain insert and search operations with example. What are its applications?	7 M
5	Explain Binomial Heaps. What is a binomial tree B_k ? List properties of Binomial Heap.	7 M

6	Compare: Binary Heap, Binomial Heap, and Fibonacci Heap — time complexities for key operations.	7 M
7	Why are Red-Black Trees preferred over AVL Trees? What is the difference in their balancing strategy?	5 M
8	What is black-height of a Red-Black Tree? Show an example RB Tree and compute black-height of each node.	5 M

■ Your 10-Day Study Schedule — Honest & Prioritized

My honest recommendation on time allocation:

Unit 1 (Trees) and Unit 3 (DP) need the most time because they are numerical-heavy. Unit 2 (DBMS) is conceptual but normalization takes practice. Unit 4 (Backtracking) is learnable in 2 days if you focus on diagrams. Unit 5 (Advanced DS) — do NOT sink more than 1.5 days into it. The pattern I've identified from your slides: Tree numericals + DP table-filling + Backtracking trees = at least 50% of your exam marks.

Day 1	Unit 1 – Tree Basics + Traversals	Learn terminology. Practice 6 traversal problems. Master constructing tree from Inorder+Preorder. Do the H K D B I L E A example from slides.	5 hrs
Day 2	Unit 1 – BST + AVL	Insert elements into BST with diagram. All 3 BST deletion cases. AVL rotations LL/LR/RL/RR. Build AVL for: a,z,b,y,c,x,d,w,e,v,f.	5 hrs
Day 3	Unit 1 – B-Tree + Heap Sort	B-Tree of order 4: insertions from slides. Heap Sort: BUILD-HEAP + HEAPIFY algorithm. Trace on an array. Full Binary Tree theorem formulas.	4 hrs
Day 4	Unit 2 – Keys + Constraints + SQL	All key types with definitions + examples. SQL constraints. Aggregate functions with GROUP BY + HAVING. WHERE vs HAVING distinction.	4 hrs
Day 5	Unit 2 – FDs + Normalization	HARDEST conceptual day. Functional Dependencies, attribute closure. 1NF→2NF→3NF→BCNF with 3 worked numericals. Canonical cover algorithm.	5 hrs
Day 6	Unit 3 – DP: Knapsack + LCS	Practice filling Knapsack table for 3 different problems. LCS table for 2 string pairs. Memorise the recurrence relations. Time complexities.	5 hrs
Day 7	Unit 3 – DP: Floyd Warshall + MCM + Assembly Line	Floyd-Warshall on a 4-5 vertex graph. Matrix Chain Multiplication for 3-4 matrices. Assembly Line Scheduling formula.	5 hrs
Day 8	Unit 4 – Backtracking	4-Queen and 8-Queen solution with state-space tree. Graph Coloring: 4-vertex example. Subset Sum $S=\{3,4,5,6\}$, $X=9$. Hamiltonian Cycle definition.	4 hrs
Day 9	Unit 4 B&B; + Unit 5	TSP using Branch and Bound: cost matrix reduction, node expansion. Then Unit 5: RB Tree 5 properties + height theorem. B+ vs B-Tree. Tries basics.	5 hrs
Day 10	Full Revision + Mock	Attempt 2 previous year papers timed. Mark all weak areas. Redo those specific numericals. Focus on formulae sheet (next section).	5 hrs

Days 5 and 6 (highlighted) are your highest-effort days — do not skip or shortcut them.

■ Formulae & Quick-Reference Card — Stick This on Your Wall

Max nodes at level l	2^l (root is level 0)
Max nodes in BT height h	$2^{(h+1)} - 1$
Min height for N nodes	$\text{floor}(\log_2(N+1)) - 1$
Full BT: Leaves vs Internal	$L = l+1 \mid N = 2l+1 \mid N = 2L-1$
Heap Sort complexity	$O(n \log n)$ — worst, average, best all same
AVL Balance Factor	$BF = \text{height}(\text{left}) - \text{height}(\text{right}) \mid \text{Valid: } -1, 0, +1$
B-Tree of order m (non-root)	Min keys: $\text{ceil}(m/2)-1$ Max keys: $m-1$
1NF	All attributes atomic — no repeating groups
2NF	1NF + No partial dependency on primary key
3NF	2NF + No transitive dependency
BCNF	For every $X \rightarrow Y$, X must be a superkey
Canonical Cover steps	Step 1: Decompose RHS. Step 2: Remove extraneous LHS. Step 3: Remove redundant FDs
0/1 Knapsack recurrence	$c[i,w] = \max(c[i-1,w], v_i + c[i-1, w-w_i])$ if $w_i \leq w$, else $c[i-1,w]$
LCS recurrence	$L[i,j] = L[i-1,j-1]+1$ if $X_i=Y_j$, else $\max(L[i-1,j], L[i,j-1])$
Floyd-Warshall update	$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$ for each k
MCM complexity	$O(n^3)$ — 3 nested loops over all subchains
N-Queens state space	State-space tree built via DFS. Backtrack when conflict found.
RB Tree height bound	$h \leq 2\log_2(n+1)$ where n = number of internal nodes
RB Tree: 5 properties	(1) Red/Black (2) Root black (3) Leaves black (4) Red $\rightarrow$ black children (5) Equal black-height paths
Black-height bh(x)	Number of black nodes from x to leaf, NOT counting x itself

■ Scoring Strategy — My Honest Advice for You

■ Warning — where most students lose marks:

Units 3 and 5 are where students waste time memorising without practising. DP algorithms CANNOT be memorised passively — you have to fill tables by hand at least 5 times each. Similarly, RB Tree insertion details are very hard to reproduce without practice. If you have limited time, skip deep RB insertion and focus on properties + height theorem.

■ Where you can guarantee marks with 2 days of work:

Unit 1 tree traversals + BST operations (10+ marks almost certain). Unit 2 normalization — if you learn the 3-step algorithm to go $1NF \rightarrow 2NF \rightarrow 3NF \rightarrow BCNF$ with one worked example, you can answer 10 marks confidently. Unit 4 N-Queens — the 4-queen solution is the SAME every time. Draw it once, memorise it.

■ How to structure 10-mark answers:

Always follow: (1) Definition in 2 lines. (2) Properties/rules as bullet points. (3) Algorithm or recurrence. (4) Worked numerical example with diagram. (5) Time complexity. This structure alone earns 8/10 even with partial knowledge. Examiners reward structure.

■ Time management in the exam hall:

Attempt Unit 1 (Trees) first — it is the most familiar. Then Unit 2 (DBMS) normalization. Then whichever DP problem you practiced most. Do NOT attempt Unit 5 deep questions unless you have genuinely revised them — a partially correct Unit 4 answer (backtracking tree) scores more than a Unit 5 guess.

■ The #1 thing that will separate you from others:

Diagrams. Every answer in this subject — AVL rotations, DP tables, state-space trees, RB tree examples, B-tree insertions — rewards clear, labelled diagrams. Write less prose, draw more diagrams. Examiners mark on sight — a good diagram gets marks instantly.

■ Good luck — you now have everything you need. Work through the numericals daily and you will score well.